

Cloud Configuration Management avec Terraform

HashiCorp-Terraform



INFRASTRUCTURE

 Terraform

 Packer

NETWORKING

 Consul

SECURITY

 Vault

 Boundary

APPLICATIONS

 Nomad

 Waypoint

 Vagrant

Sommaire

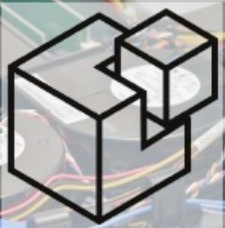
- Infrastructure as code
- Terraform
- Première ressource avec Terraform
- Outputs
- Variables & Interpolation
- Modules
- Provisioners
- Meta-arguments
- Datasources
- State Management
- Manager les ressources AWS avec Terraform



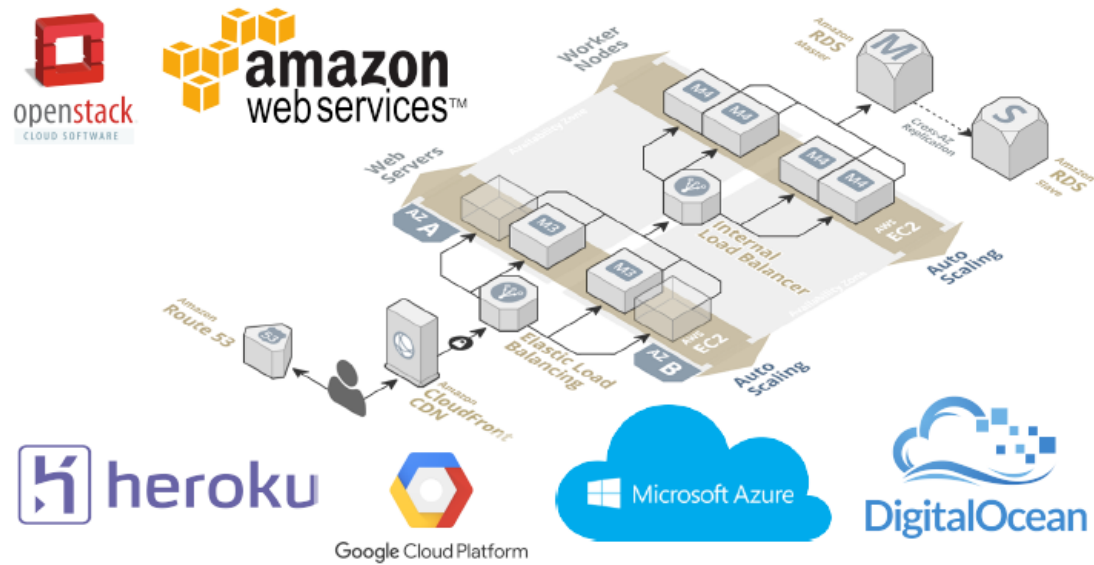
Infrastructure as code



From Servers ...



...to Services



Infrastructure as code (IaC) - Besoin

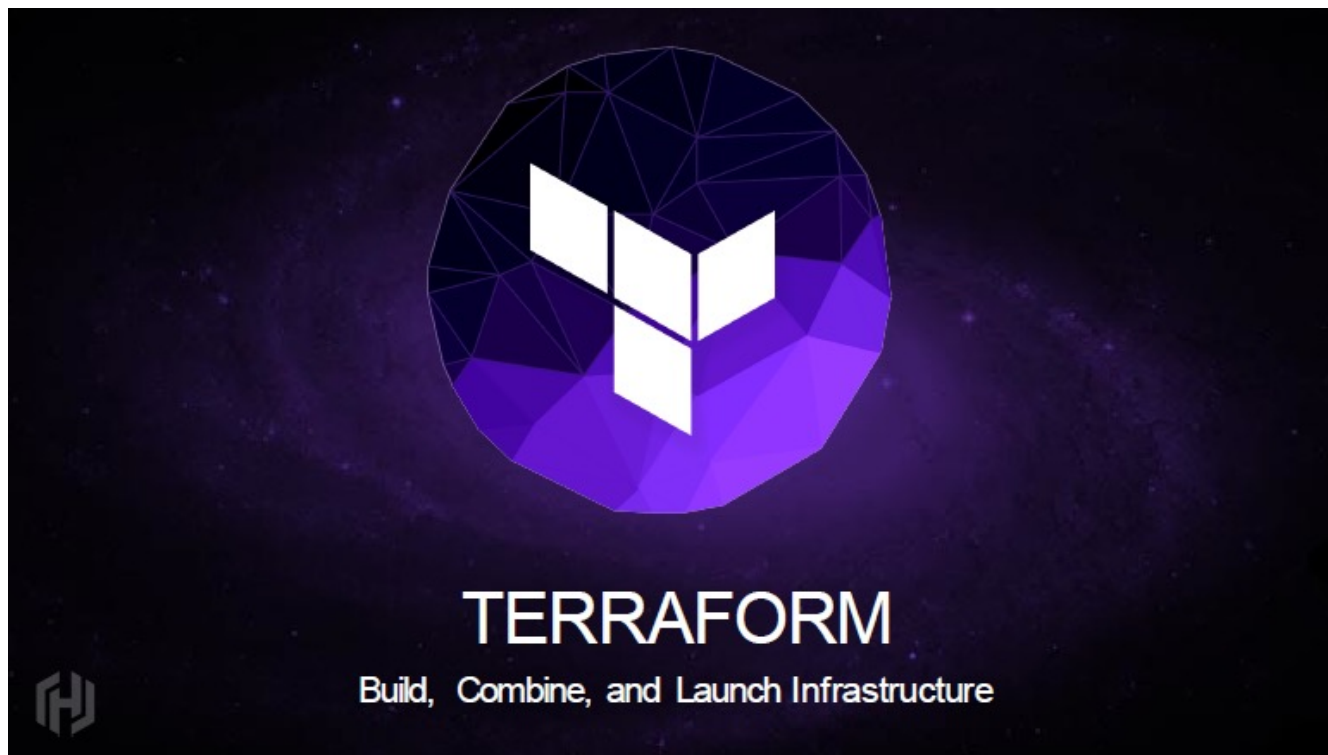
- L'infrastructure en tant que code (IaC) est l'automatisation de la gestion de l'infrastructure en utilisant des principes et des pratiques de développement logiciel.
- L'idée est que vous traitez votre infrastructure comme un logiciel, puis que vous écrivez, testez et exécutez du code pour définir, déployer, mettre à jour et détruire votre infrastructure.
- Vous écrivez du code pour gérer vos serveurs, bases de données, réseaux, journaux, déploiement et configuration d'applications.
- Lorsque vous souhaitez apporter des modifications à votre infrastructure, vous apportez des modifications au code, vous le testez, puis vous l'appliquez à vos systèmes.

Défis de laC

- Besoin d'apprendre à coder !!
- Je ne connais pas l'impact du changement !!
- Besoin d'annuler le changement, !!
- Impossible de suivre les modifications !!
- Impossible d'automatiser une ressource !!
- Environnements multiples pour l'infrastructure !!

Défis de IaC

➔ Terraform a été créé pour résoudre ces défis.





Terraform



Qu'est-ce que Terraform?

- Terraform est une infrastructure open-source en tant qu'outil de code développé par HashiCorp.
- Il est utilisé pour définir et provisionner l'infrastructure complète à l'aide d'un langage déclaratif facile à maîtriser.
- Il s'agit d'un outil de provisioning d'infrastructure dans lequel vous pouvez stocker la configuration de votre infrastructure cloud sous forme de codes.

Avantages de Terraform

- Fait de l'orchestration, pas seulement de la gestion de la configuration
- Prend en charge plusieurs fournisseurs tels que AWS, Azure, GCP, DigitalOcean et bien d'autres
- Fournir une infrastructure immuable où la configuration change en douceur
- Utilise un langage facile à comprendre, HCL (langage de configuration HashiCorp)
- Facilement portable vers n'importe quel autre fournisseur
- Prend en charge l'architecture client uniquement, donc pas besoin de gestion de configuration supplémentaire sur un serveur
- Langage facile à lire, Extensible et Gratuit

Concepts de base de Terraform (1/2)

- **Provider:** C'est un plugin pour interagir avec les API d'un fournisseur de ressources.
- **Resource:** Il s'agit d'un bloc d'un ou plusieurs objets d'infrastructure (instances de calcul, réseaux virtuels, etc.), qui sont utilisés dans la configuration et la gestion de l'infrastructure.
- **Module:** C'est un dossier avec des modèles Terraform où toutes les configurations sont définies, pour des objectifs de partage et réutilisabilité
- **Variables:** il s'agit d'une paire clé-valeur utilisée par les modules Terraform pour permettre la personnalisation de l'infrastructure.

Concepts de base de Terraform (2/2)

- **Data Source:** Il est implémenté par les fournisseurs pour renvoyer des informations sur les objets externes à terraform.
- **Output:** Ce sont les valeurs de retour d'un module terraform qui peuvent être utilisées par d'autres configurations.
- **Plan:** C'est l'une des étapes où il détermine ce qui doit être créé, mis à jour ou détruit pour passer de l'état réel / actuel de l'infrastructure à l'état souhaité.
- **Apply:** C'est l'une des étapes où il applique les changements d'état réel / actuel de l'infrastructure afin de passer à l'état souhaité.

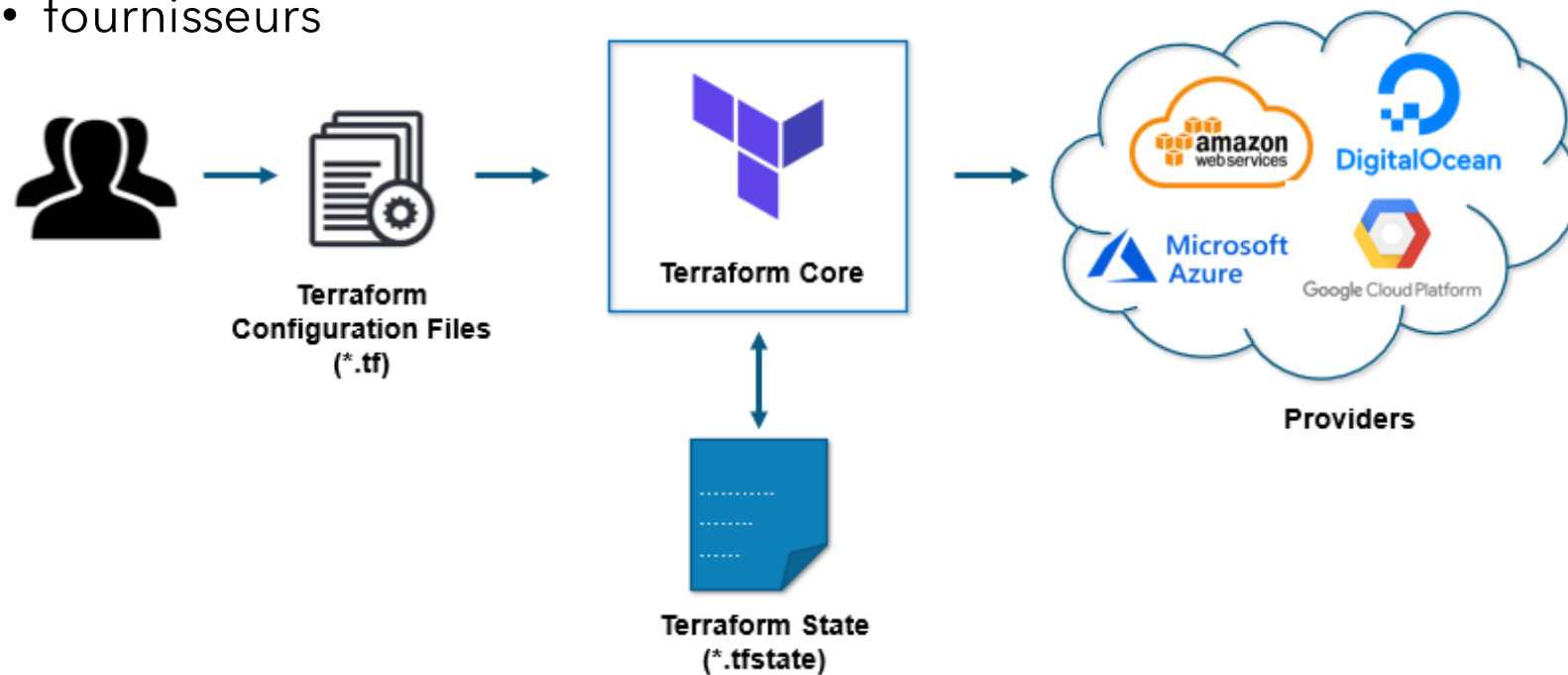
Cycle de vie Terraform



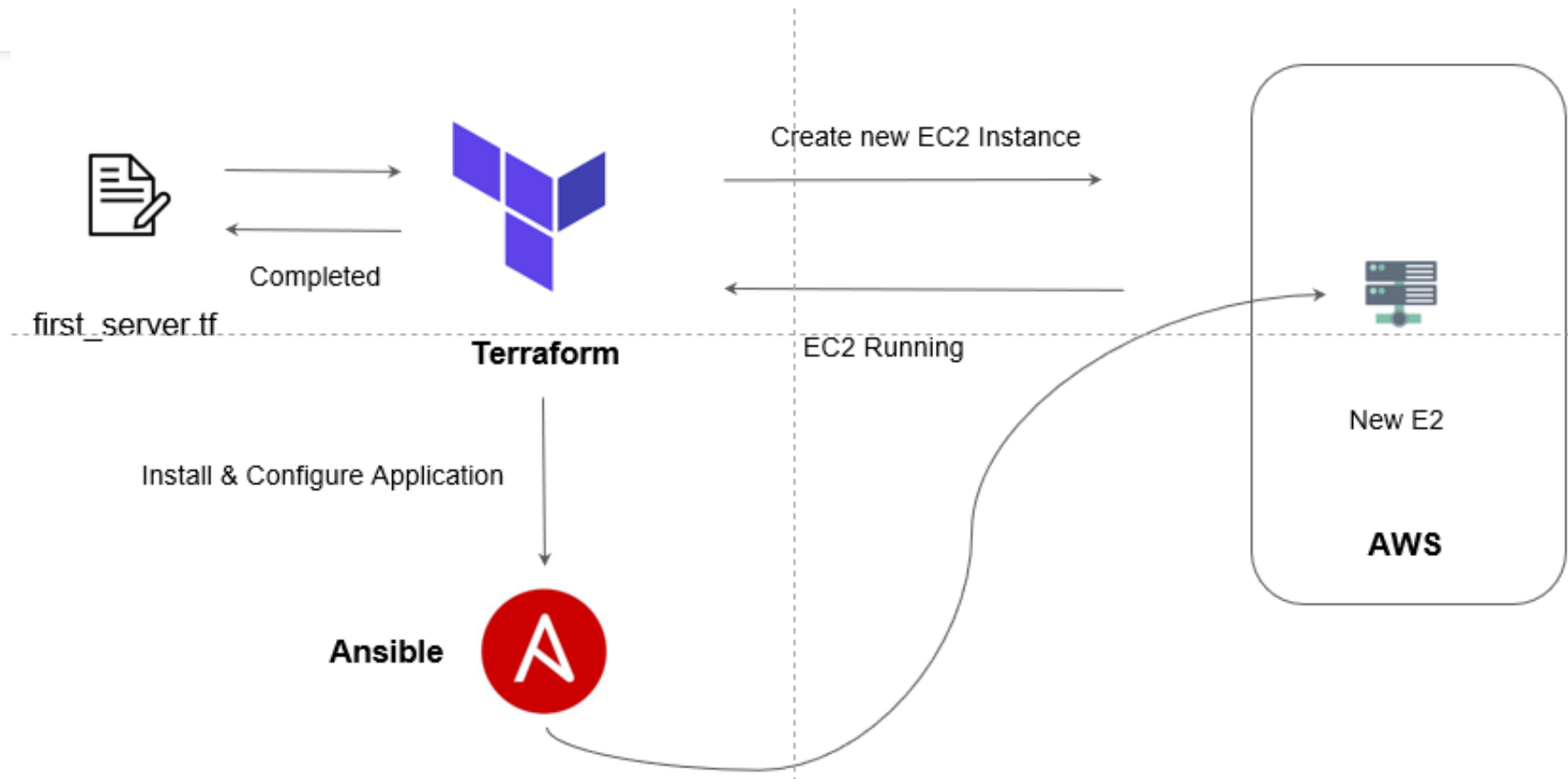
- **terraform init**: initialise le répertoire de travail qui comprend tous les fichiers de configuration
- **terraform plan**: est utilisé pour créer un plan d'exécution pour atteindre l'état souhaité de l'infrastructure. Les changements dans les fichiers de configuration sont effectués afin d'atteindre l'état souhaité.
- **terraform apply**: effectue ensuite les modifications de l'infrastructure telles que définies dans le plan, et l'infrastructure arrive à l'état souhaité.
- **terraform destroy**: est utilisé pour supprimer toutes les ressources d'infrastructure créées après la phase d'application

Fonctionnement de Terraform

- Terraform a deux composants principaux qui composent son architecture:
 - Terraform Core
 - fournisseurs



IaC et Configuration Management



Installation

Windows

macOS

Linux

FreeBSD

OpenBSD

Solaris



Atelier

- Lab 1:
- Installation Terraform



Déployez vos
premières ressources



Resource & Providers

- Terraform s'appuie sur des plugins appelés « provider » pour interagir avec les systèmes distants (AWS, GCP, AZURE, Digital Ocean , ...), .
- Les configurations Terraform doivent déclarer les providers dont elles ont besoin, afin que Terraform puisse les installer et les utiliser.
- Certains providers nécessitent une configuration (comme des URL de point de terminaison ou des régions cloud) avant de pouvoir être utilisés.
- Une configuration de fournisseur est créée à l'aide d'un bloc **provider** :

```
provider "gpc" {  
    project = "acme-app"  
    region  = "us-central1"  
}
```

Resource & Providers

- Les **ressources** sont l'élément le plus important du langage Terraform.
- Chaque bloc de ressources décrit un ou plusieurs objets d'infrastructure, tels que des réseaux virtuels, des instances de calcul ou des composants de niveau supérieur tels que des enregistrements DNS.
 - **Resource Blocks**: déclaration des ressources.
 - **Resource Behavior**: explique plus en détail comment Terraform gère les déclarations de ressources lors de l'application d'une configuration.
 - **La section meta-arguments**: des arguments spéciaux qui peuvent être utilisés avec tous les types de ressources, y compris `depends_on`, `count`, `for_each`, `provider` et `lifecycle`.
 - **Les drivers**: des documents configurant les actions de post-crédation pour une ressource à l'aide des blocs `provisioner` et `connection`.

Blocs de ressources

- Un bloc de ressource déclare une ressource d'un type donné (eg. "aws_instance") avec un nom local donné (eg. "web").
- Le nom est utilisé pour faire référence à cette ressource ailleurs dans le même module Terraform, mais n'a aucune signification en dehors de la portée de ce module.

```
resource "aws_instance" "web" {  
    ami = "ami-a1b2c3d4"  
    instance_type = "t2.micro"  
}
```

- Dans le corps du bloc (entre {et }) se trouvent les arguments de configuration de la ressource elle-même.
- Chaque ressource est associée à un type de ressource unique , qui détermine le type d'objet d'infrastructure qu'elle gère et les arguments et autres attributs pris en charge par la ressource.



Atelier

- Lab 2 :
Première infrastructure AWS avec
Terraform



Outputs



Les valeurs de sortie (Outputs)

- Rendre les informations sur l'infrastructure disponibles pour la CLI
- Exposer des informations pour d'autres configurations Terraform.
 - Similaires aux valeurs de retour dans les langages de programmation.
- Les valeurs de sortie ont plusieurs utilisations :
 - Un module enfant peut utiliser des sorties pour exposer un sous-ensemble de ses attributs de ressource à un module parent.
 - Un module racine peut utiliser des sorties pour imprimer certaines valeurs dans la sortie CLI après avoir exécuté « terraform apply ».
 - Lors de l'utilisation de l'état distant, les sorties du module racine sont accessibles par d'autres configurations via une source de données (datasource) appelée « terraform_remote_state »

Les valeurs de sortie (Output)

- Chaque valeur de sortie exportée par un module doit être déclarée à l'aide d'un bloc output

```
output "instance_ip_addr" {  
    value = aws_instance.server.private_ip  
    description = "The private IP address of the main server instance."  
}
```

- L'argument value prend une expression dont le résultat doit être retourné à l'utilisateur.
- Vous pouvez faire référence à n'importe quel attribut en spécifiant les éléments suivants dans votre variable :
 - Le type de la ressource : eg. aws_instance
 - Le nom de la ressource : eg. web_server
 - Le nom de l'attribut : eg. public_ip

Les valeurs de sortie (Output)

- Vous pouvez également utiliser les attributs dans un script :

```
ressource "aws_instance" "example" {  
    ami = "${lookup(var.AMIS, var.AWS_REGION)}"  
    instance_type = "t2.micro"  
    provisioner "local-exec" {  
        command = "echo ${aws_instance.example.private_ip} >> private_ips.txt"  
    }  
}
```

- Utile par exemple pour lancer des scripts d'automatisation après le provisionnement de l'infrastructure
- Vous pouvez remplir les adresses IP dans un fichier inventaire Ansible
- Ou autre possibilité : exécuter un script (avec des attributs en argument) qui se chargera d'un mappage du nom de la ressource et de l'adresse IP



Atelier

- Lab 3 :
- Les outputs

Déploiements dynamiques avec les variables

- Le langage HCL de Terraform comprend quelques types de blocs pour préparer des déploiements dynamiques avec des variables.
 - Les variables d'entrée (Input Variables): servent de paramètres pour un module Terraform, afin que les utilisateurs puissent personnaliser le comportement sans modifier la source.
 - Les valeurs de sortie (Output Values): sont comme les valeurs de retour pour un module Terraform.
 - Les valeurs locales (Local Values): sont une fonctionnalité pratique pour attribuer un nom court à une expression.
- Personnalisation de votre déploiement ➔ Attributs/variables
- Récupération d'informations dynamiques ➔ output

Les variables d'entrée (Input Variables)

- Tout dans un seul fichier n'est pas génial
- Utiliser des variables pour masquer les secrets
 - Vous ne voulez pas les informations d'identification AWS dans votre référentiel git
- Utiliser des variables pour les éléments susceptibles de changer
 - Les AMI sont différents selon la région
- Utiliser des variables pour faciliter vous-même la réutilisation des fichiers terraform

Les variables d'entrée (Input Variables)

- Permet de partager des modules sur différentes configurations Terraform, rendant votre module composable et réutilisable.
 - Lorsque vous déclarez des variables dans le module racine de votre configuration, vous pouvez définir leurs valeurs à l'aide des options CLI et des variables d'environnement.
 - Lorsque vous les déclarez dans les modules enfants , le module appelant doit passer des valeurs dans le module bloc.

Les variables d'entrée (Input Variables)

- Chaque variable d'entrée acceptée par un module doit être déclarée à l'aide d'un bloc de variable
- Terraform définit des arguments facultatifs pour les déclarations de variables :
 - **default** - Une valeur par défaut qui rend alors la variable facultative.
 - **type** - Cet argument spécifie quels types de valeur sont acceptés pour la variable.
 - **validation** - Un bloc pour définir des règles de validation, généralement en plus des contraintes de type.
 - **sensitive** - Limite la sortie de l'interface utilisateur Terraform lorsque la variable est utilisée dans la configuration.

```
variable "image_id" {  
    type = string  
}  
variable "image_id" {  
    type = string  
    description = "The id ... "  
    validation {  
        condition = length(var.image_id) > 4  
        && substr(var.image_id, 0, 4) == "ami-"  
        error_message = "The image_id value must be a  
        valid AMI id, starting with \"ami-\" "."  
    }  
}  
variable "docker_ports" {  
    type = list(object({  
        internal = number  
        external = number  
        protocol = string  
    }))  
    default = [ {  
        internal = 8300  
        external = 8300  
        protocol = "tcp"  
    },  
    ]  
}
```

Les variables d'entrée (exemple)

- Soit la configuration Terraform suivante:

example.tf

```
provider "aws" {  
    access_key      = "ACCESS_KEY_HERE"  
    secret_key      = "SECRET_KEY_HERE"  
    region          = "us-east-1"  
}  
  
resource "aws_instance" "example" {  
    ami              = "ami-0d729a60"  
    instance_type    = "t2.micro"  
}
```

Les variables d'entrée (exemple)

provider.tf

```
provider "aws" {  
  access_key = "${var.AWS_ACCESS_KEY}"  
  secret_key = "${var.AWS_SECRET_KEY}"  
  region     = "${var.AWS_REGION}"  
}
```

vars.tf

```
variable "AWS_ACCESS_KEY" {}  
variable "AWS_SECRET_KEY" {}  
variable "AWS_REGION" {  
  default = "eu-west-1"  
}
```

terraform.tfvars

```
AWS_ACCESS_KEY = ""  
AWS_SECRET_KEY = ""  
AWS_REGION = ""
```

instance.tf

```
resource "aws_instance" "example" {  
  ami           = "ami-0d729a60"  
  instance_type = "t2.micro"  
}
```

Les variables d'entrée (exemple)

provider.tf

```
provider "aws" {  
  access_key = "${var.AWS_ACCESS_KEY}"  
  secret_key = "${var.AWS_SECRET_KEY}"  
  region     = "${var.AWS_REGION}"  
}
```

instance.tf

```
resource "aws_instance" "example" {  
  ami      = "${lookup(var.AMIS, var.AWS_REGION)}"  
  instance_type = "t2.micro"  
}
```

vars.tf

```
variable "AWS_ACCESS_KEY" {}  
variable "AWS_SECRET_KEY" {}  
variable "AWS_REGION" {  
  default = "eu-west-1"  
}  
variable "AMIS" {  
  type = "map"  
  default = {  
    us-east-1 = "ami-13be557e"  
    us-west-2 = "ami-06b94666"  
    eu-west-1 = "ami-0d729a60"  
  }  
}
```

<https://cloud-images.ubuntu.com/locator/ec2/>

Les valeurs locales (Local Values)

- Des valeurs nommées auxquelles vous pouvez vous référer dans votre configuration.
- Permet de simplifier la configuration Terraform et éviter les répétitions.

```
locals {  
    service_name = "forum"  
    owner        = "Community Team"  
}  
  
locals {  
    instance_ids = concat(aws_instance.blue.*.id,  
        aws_instance.green.*.id)  
}  
  
locals {  
    common_tags = {  
        Service = local.service_name  
        Owner   = local.owner  
    }  
}
```

Types de variables Terraform

- Les variables Terraform ont été complètement retravaillées depuis Terraform 0.12
- Vous pouvez maintenant avoir plus de contrôle sur les variables, et avoir des boucles *for* et *for-each*, qui n'étaient pas possibles avec les versions antérieures
- Vous n'êtes pas obligé de spécifier le type dans les variables, mais il est recommandé

Types de variables Terraform

- Types simples:

- String
- Number
- Bool

```
variable "a-string" {  
    type = string  
}  
variable "this-is-a-number" {  
    type = number  
}  
variable "true-or-false" {  
    type = bool  
}
```

Types de variables Terraform

- Types complexes:
 - List(type)
 - Set(type)
 - Map(type)
 - Object({<ATTR NAME> = <TYPE>, ... })
 - Tuple([<TYPE>, ...])

Types de variables Terraform

- List et Map
 - List : [0,1,5,2]
 - Map : {"clé" = "valeur"}
- L'ordre de définition des valeurs d'une liste ne change pas, elle renverra toujours 0,1,5,2 et non 5,1,2,0
- Un "**Set**" est comme une liste, mais il ne conserve pas l'ordre dans lequel vous l'avez mis et ne peut que contenir des valeurs uniques
- Une liste qui a [5, 1, 1, 2] devient [1,2,5] dans un Set (lorsque vous l'affichez, terraform va le trier)

Types de variables Terraform

- Un **object** est comme une **Map**, mais chaque élément peut avoir un type différent
- Par exemple:

```
variable "an-object" {  
  type = "object"  
  default = {  
    firstname = "John"  
    housenumber = 10  
  }  
}
```

Types de variables Terraform

- Un **tuple** est comme une liste, mais chaque élément peut avoir un type différent
- Par exemple:

```
variable "a-tuple" {  
  type = "tuple"  
  default = [ 0, "string", false ]  
}
```

Types de variables Terraform

- Les types les plus courants sont **list** et **map**, les autres ne sont utilisés que sporadiquement
- Ceux dont vous devez vous souvenir sont les types de variables simples **string**, **nombre**, **bool**, et **list** et **map**
- Vous pouvez laisser Terraform décider le type de la variable:

```
variable "a-string" {  
    default = "this is a string"  
}  
variable "this-is-a-list" {  
    default = [ "list of", "strings" ]  
}
```



Atelier

- Lab 4 :
Déploiements dynamiques avec les variables



Les modules



Les modules

- Vous pouvez utiliser des modules pour rendre votre projet terraform plus organisé
- Utilisez des modules tiers
 - Modules sur Github
- Réutilisez des parties de votre code
 - par exemple, pour configurer le réseau dans AWS - le réseau privé virtuel (VPC)

Les modules

- Utiliser un module sur github

```
module "module-example" {  
  source = "github.com/benbahri/terraform-module-example"  
}
```

- Utiliser un module sur répertoire local

```
module "module-example" {  
  source = "./module-example"  
}
```

Les modules

- Passer des arguments à un module

main.tf

```
module "module-example" {  
  source = "../module-example"  
  region = "us-west-1"  
  ip-range = "10.0.0.0/8"  
  cluster-size = "3"  
}
```

Les modules

Dans le dossier du module, vous avez à nouveau des fichiers terraform :

module-example/vars.tf

```
variable "region" {}  
variable "ip-range" {}  
variable "cluster-size" {}
```

module-example/cluster.tf

```
# vars can be used here  
resource "aws_instance" "instance-1" { ... }  
resource "aws_instance" "instance-2" { ... }  
resource "aws_instance" "instance-3" { ... }
```

module-example/output.tf

```
output "aws-cluster" {  
  value = "${aws_instance.instance-1.public_ip},${aws_instance.instance-2.public_ip},  
    ${aws_instance.instance-2.public_ip}"  
}
```

Modules

- Utilisez les outputs du module dans la partie principale de votre code :

```
output "some-output" {  
  value = "${module.module-example.aws-cluster}"  
}
```

- vous pouvez utiliser les variables n'importe où dans le code terraform



Atelier

- Lab 5 :
- Les modules



Provisioners



Les provisionneurs

- Il existe 2 façons de provisionner des logiciels sur vos instances
 - Vous pouvez créer votre propre AMI personnalisée et regrouper votre logiciel avec l'image
 - **Packer** est un excellent outil pour ce faire
 - Une autre méthode consiste à démarrer des AMI standardisées, puis à y installer les logiciels, vous avez besoin de:
 - files upload
 - Utilisation de l'exécution à distance (remote exec)
 - Utilisation d'outils d'automatisation tels que chef, puppet, ansible

Les provisionneurs

- Les provisionneurs peuvent être utilisés pour modéliser des actions spécifiques sur la machine locale ou sur une machine distante afin de préparer des serveurs ou d'autres objets d'infrastructure pour le service.

Les provisionneurs - Types

- Il existe de nombreux provisionneurs disponibles
 - Le provisionneur **local-exec** : appelle un exécutable local après la création d'une ressource.
 - Le provisionneur **remote-exec** : permet de lancer un script sur une ressource distante une fois qu'elle est créée. Nous pouvons fournir une liste de commandes qui sont exécutées dans l'ordre où elles sont fournies.
 - Le provisionneur **file** : est utilisé pour copier des fichiers ou des répertoires de la machine exécutant Terraform vers la ressource nouvellement créée. Il prend également en charge les connexions de type SSH et WinRM.

Le provisionneur local-exec

- Le provisionneur local-exec invoque un exécutable local après la création d'une ressource. Cela appelle un processus sur la machine exécutant Terraform, pas sur la ressource.

```
resource "aws_instance" "web" {  
  # ...  
  provisioner "local-exec" {  
    command = "echo ${self.private_ip} >> private_ips.txt"  
  }  
}
```

- Les arguments suivants sont pris en charge :
 - **command** (Obligatoire): Il s'agit de la commande à exécuter.
 - **working_dir** (Facultatif): spécifie le répertoire de travail où *command* sera exécuté.
 - **interpreter** (Facultatif): une liste d'arguments de l'interpréteur utilisés pour exécuter *command*.
 - **environment** (Facultatif): bloc de paires clé-valeur représentant l'environnement de la commande exécutée.

Le provisionneur remote-exec

- Le provisionneur **remote-exec** invoque un script sur une ressource distante après sa création.
- Cela peut être utilisé pour exécuter un outil de gestion de configuration, amorcer un cluster, etc.
- Les arguments suivants sont pris en charge :
 - **inline** une liste de chaînes de commande. Elles sont exécutées dans l'ordre où elle sont fournis.
 - **script** Il s'agit d'un chemin (relatif ou absolu) vers un script local qui sera copié sur la ressource distante puis exécuté.
 - **scripts** Il s'agit d'une liste de chemins (relatifs ou absolus) vers des scripts locaux qui seront copiés sur la ressource distante puis exécutés. Ils sont exécutés dans l'ordre où ils sont fournis. Cela ne peut pas être fourni avec **inline** ou **script**.

```
resource "aws_instance" "web" {  
  # ...  
  
  provisioner "remote-exec" {  
    inline = [  
      "puppet apply",  
      "consul join ${aws_instance.web.private_ip}",  
    ]  
  }  
}
```

Le provisionneur file

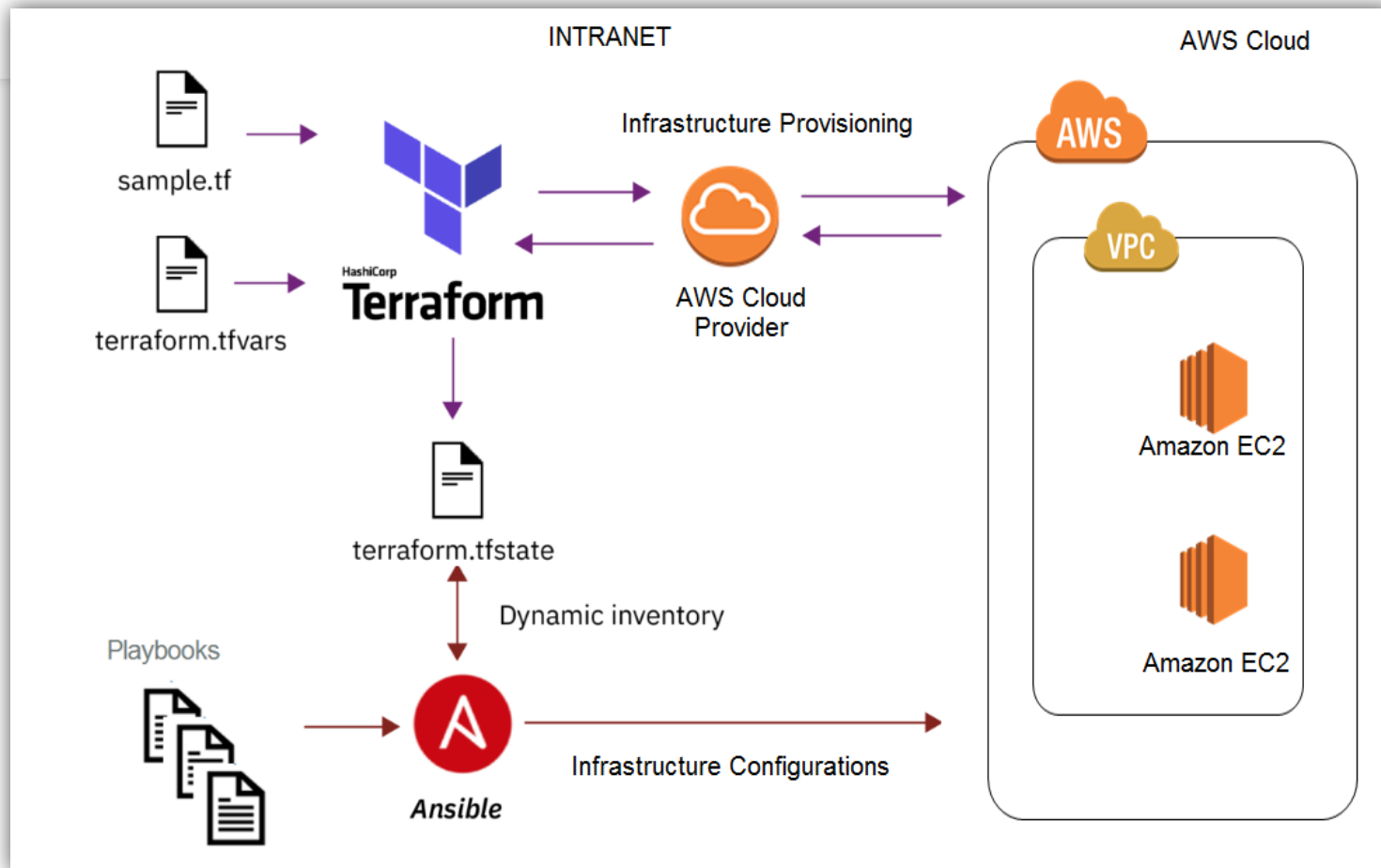
- Utilisé pour copier des fichiers ou des répertoires de la machine exécutant Terraform vers la ressource nouvellement créée.
 - Les arguments suivants sont pris en charge :
 - **source** Il s'agit du fichier ou dossier source. Cet attribut ne peut pas être spécifié avec content.
 - **content** C'est le contenu à copier sur la destination. Si la destination est un fichier, le contenu sera écrit sur ce fichier, dans le cas d'un répertoire, un fichier nommé **tf-file-content** est créé.
 - **destination** (Obligatoire) Il s'agit du chemin de destination. Il doit être spécifié en tant que chemin absolu.
- ```
resource "aws_instance" "web" {
 # ...
 # Copies the myapp.conf file to /etc/myapp.conf
 provisioner "file" {
 source = "conf/myapp.conf"
 destination = "/etc/myapp.conf"
 }

 # Copies the string in content into /tmp/file.log
 provisioner "file" {
 content = "ami used: ${self.ami}"
 destination = "/tmp/file.log"
 }
}
```

# Les provisionneurs

- État actuel de terraform avec l'automatisation :
  - **Chef** est intégré à terraform, vous pouvez ajouter des déclarations Chef
  - Vous pouvez exécuter l'agent de **Puppet** à l'aide de **remote-exec**
  - Pour Ansible, vous pouvez d'abord exécuter « terraform apply » et générer les adresses IP, puis exécuter ansible-playbook sur ces hôtes
    - Cela peut être automatisé dans un script de workflow
    - Il existe des initiatives tierces intégrant Ansible avec terraform

# Les provisionneurs





## Atelier

- Lab 6 :
- Les provisioners



# Commandes Terraform



# Commandes Terraform

- Terraform est très axé sur les définitions de ressources
- Il dispose d'un ensemble d'outils limité pour modifier, importer, créer ces définitions de ressources
- On utilise ces commandes au fur et à mesure selon le besoin

# Commandes Terraform

| Commande                    | Description                                                                                                                                  |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| terraform apply             | Applies state                                                                                                                                |
| destroy                     | Destroys all terraform managed state (use with caution)                                                                                      |
| fmt                         | Rewrite terraform configuration files to a canonical format and style                                                                        |
| get                         | Download and update modules                                                                                                                  |
| graph                       | Create a visual representation of a configuration or execution plan                                                                          |
| import [options] ADDRESS ID | Import will try and find the infrastructure resource identified with ID and import the state into terraform.tfstate with resource id ADDRESS |
| output [options] [NAME]     | Output any of your resources. Using NAME will only output a specific resource                                                                |
| plan                        | terraform plan, show the changes to be made to the infrastructure                                                                            |

# Commandes Terraform

| Commande | Description                                                                                                                                              |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| push     | Push changes to Atlas, Hashicorp's Enterprise tool that can automatically run terraform from a centralized server                                        |
| refresh  | Refresh the remote state. Can identify differences between state file and remote state                                                                   |
| remote   | Configure remote state storage                                                                                                                           |
| show     | Show human readable output from a state or a plan                                                                                                        |
| state    | Use this command for advanced state management, e.g. Rename a resource with <code>terraform state mv aws_instance.example aws_instance.production</code> |
| taint    | Manually mark a resource as tainted, meaning it will be destructed and recreated at the next apply                                                       |
| validate | validate your terraform syntax                                                                                                                           |
| untaint  | undo a taint                                                                                                                                             |



# Atelier

- Lab 7:  
Terraform Commands



# Interpolation



# Interpolation

- Dans Terraform, vous pouvez interpoler d'autres valeurs en utilisant **`${...}`**
- Vous pouvez utiliser des fonctions mathématiques simples, faire référence à d'autres variables ou utiliser des structures conditionnelles (if-else)
- Interpoler:
  - **Variables** : `${var.VARIABLE_NAME}` fait référence à une variable
  - **Ressources** : `${aws_instance.name.id}` (type.resource-name.attr)
  - **Source de données** : `${data.template_file.name.rendered}`  
(data.type.resourcename.attr)

# Interpolation: variables

| Nom             | Syntaxe               | Exemple                                                                                           |
|-----------------|-----------------------|---------------------------------------------------------------------------------------------------|
| String variable | var.name              | <code>\${var.SOMETHING}</code>                                                                    |
| Map variable    | var.MAP["key"]        | 1) <code>\${var.AMIS["us-east-1"]}</code><br>2) <code>\${lookup(var.AMIS, var.AWS_REGION)}</code> |
| List variable   | var.LIST, var.LIST[i] | 1) <code>\${var.subnets[i]}</code><br>2) <code>\${join(",", var.subnets)}</code>                  |

# Interpolation: various

| Nom                 | Syntaxe            | Exemples                                                                                                                         |
|---------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Outputs of a module | module.NAME.output | <code>\${module.aws_vpc.vpcid}</code>                                                                                            |
| Count information   | count.FIELD        | When using the attribute <code>count = number</code> in a resource, you can use <code>\${count.index}</code>                     |
| Path information    | path.TYPE          | <code>path.cwd</code> (current directory)<br><code>path.module</code> (module path)<br><code>path.root</code> (root module path) |
| Meta information    | terraform.FIELD    | <code>terraform.env</code> shows active workspace                                                                                |

# Interpolation

- Math:
  - Add (+), Subtract (-), Multiply (\*), and Divide (/) for float types
  - Add (+), Subtract (-), Multiply (\*), Divide (/), and Modulo (%) for integer types
  - Exemple:  $\{2 + 3 * 4\}$  donne 14

# Conditionals

- Interpolations peuvent contenir des expressions conditionnelles (if-else)
- La syntax :

**CONDITION ? TRUEVAL : FALSEVAL**

- Exemple:

```
resource "aws_instance" "myinstance" {
 [...]
 count = "${var.env == "prod" ? 2 : 1}"
}
```

# Conditionals

- Opérateurs supportés :
  - Equality: == and !=
  - Numerical comparison: >, <, >=, <=
  - Boolean logic: &&, ||, unary !



# Atelier

- Lab 8 : Meta arguments



# Datasources



# Datasources

- Les sources de données permettent à Terraform d'utiliser des informations
  - définies en dehors de Terraform,
  - définies par une autre configuration Terraform distincte
  - ou modifiées par des fonctions.

# Datasources

- Pour certains providers (comme AWS), terraform fournit des sources de données
- Les sources de données vous fournissent des informations dynamiques
  - De nombreuses données sont disponibles par AWS dans un format structuré à l'aide de son API
  - Terraform expose également ces informations à l'aide de sources de données
- Exemples:
  - Liste des AMIs
  - Liste des zones de disponibilité
- Une source de données est accessible via un type particulier de ressource appelée **data resource**, déclarée à l'aide d'un bloc **data**

# Datasources

- Un autre exemple est la source de données qui vous donne toutes les adresses IP utilisées par AWS
- C'est très utile si vous souhaitez filtrer le trafic en fonction d'une région AWS
  - par exemple, autoriser tout le trafic des instances Amazon en Europe
- Le filtrage du trafic dans AWS peut être effectué à l'aide de groupes de sécurité
  - Le trafic entrant et sortant peut être filtré par protocole, plage IP et Port
  - Similaire à iptables (linux) ou à un pare-feu

# Datasources

```
data "aws_ip_ranges" "european_ec2" {
 regions = ["eu-west-1", "eu-central-1"]
 services = ["ec2"]
}

resource "aws_security_group" "from_europe" {
 name = "from_europe"

 ingress {
 from_port = "443"
 to_port = "443"
 protocol = "tcp"
 cidr_blocks = ["${data.aws_ip_ranges.european_ec2.cidr_blocks}"]
 }
 tags {
 CreateDate = "${data.aws_ip_ranges.european_ec2.create_date}"
 SyncToken = "${data.aws_ip_ranges.european_ec2.sync_token}"
 }
}
```





# Atelier

- Lab :
- Les datasources



Terraform state



# Terraform state

- Terraform conserve l'état remote de l'infrastructure
- Il le stocke dans un fichier appelé ***terraform.tfstate***
- Il existe également une sauvegarde de **l'état précédent** dans ***terraform.tfstate.backup***
- Lorsque vous exécutez *terraform apply*, un nouveau *terraform.tfstate* et un backup sont écrits
  - Si l'état distant change et que vous exécutez à nouveau sur *terraform apply*, terraform apporte des modifications pour atteindre à nouveau l'état distant correct
  - Eg. vous résiliez une instance gérée par terraform, après *terraform apply* elle sera recréée

# Terraform state

- Vous pouvez conserver le **terraform.tfstate** dans le contrôle de version (par exemple Git)
- Ce qui vous donne un historique de votre fichier terraform.tfstate (qui est juste un gros fichier JSON)
- Il vous permet de collaborer avec d'autres membres de l'équipe

# Terraform state

- L'état de terraform peut être sauvegardé à distance, en utilisant la fonctionnalité **backend** dans terraform
- La valeur par défaut est un backend local (le fichier d'état terraform local)
- Les autres supports incluent :
  - s3 (avec un mécanisme de verrouillage utilisant dynamoDB)
  - consul (avec verrouillage)
  - terraform enterprise (la solution commerciale)

# Terraform state

- L'utilisation de la fonctionnalité **backend** présente des avantages :
  - Travail en équipe : cela permet la collaboration, l'état distant est toujours disponible pour toute l'équipe
  - Le fichier d'état n'est pas stocké localement, donc les informations potentiellement sensibles sont stocké uniquement dans l'état distant
  - Certains backends permettent **les opérations à distance**. *terraform apply* est exécuté complètement à distance. Ceux-ci sont appelés les backends améliorés (<https://www.terraform.io/docs/backends/types/index.html>)

# Terraform state

- Il y a 2 étapes pour configurer un état distant :
  - Ajouter le code backend à un fichier .tf
  - Exécuter le processus d'initialisation

# Terraform state

- Pour configurer un repo distant consul, vous pouvez ajouter un fichier **backend.tf** avec le contenu suivant :

```
terraform {
 backend "consul" {
 address = "demo.consul.io" # hostname of consul cluster
 path = "terraform/myproject"
 }
}
```

# Terraform state

- Vous pouvez enregistrer le state dans S3:

```
terraform {
 backend "s3" {
 bucket = "mybucket"
 key = "terraform/myproject"
 region = "eu-west-1"
 }
}
```

# Terraform state

- L'utilisation d'un backend distant pour l'état de terraform vous assure de toujours avoir la dernière version de l'état
- Cela évite d'avoir à commiter et à pusher le terraform.tfstate vers le contrôle de version
- Les backends distants Terraform ne prennent pas toujours en charge le verrouillage
  - La documentation mentionne toujours si le verrouillage est disponible pour un repo distant
  - S3 et Consul le supportent



# Terraform et AWS





# Création de VPC

# Création de VPC

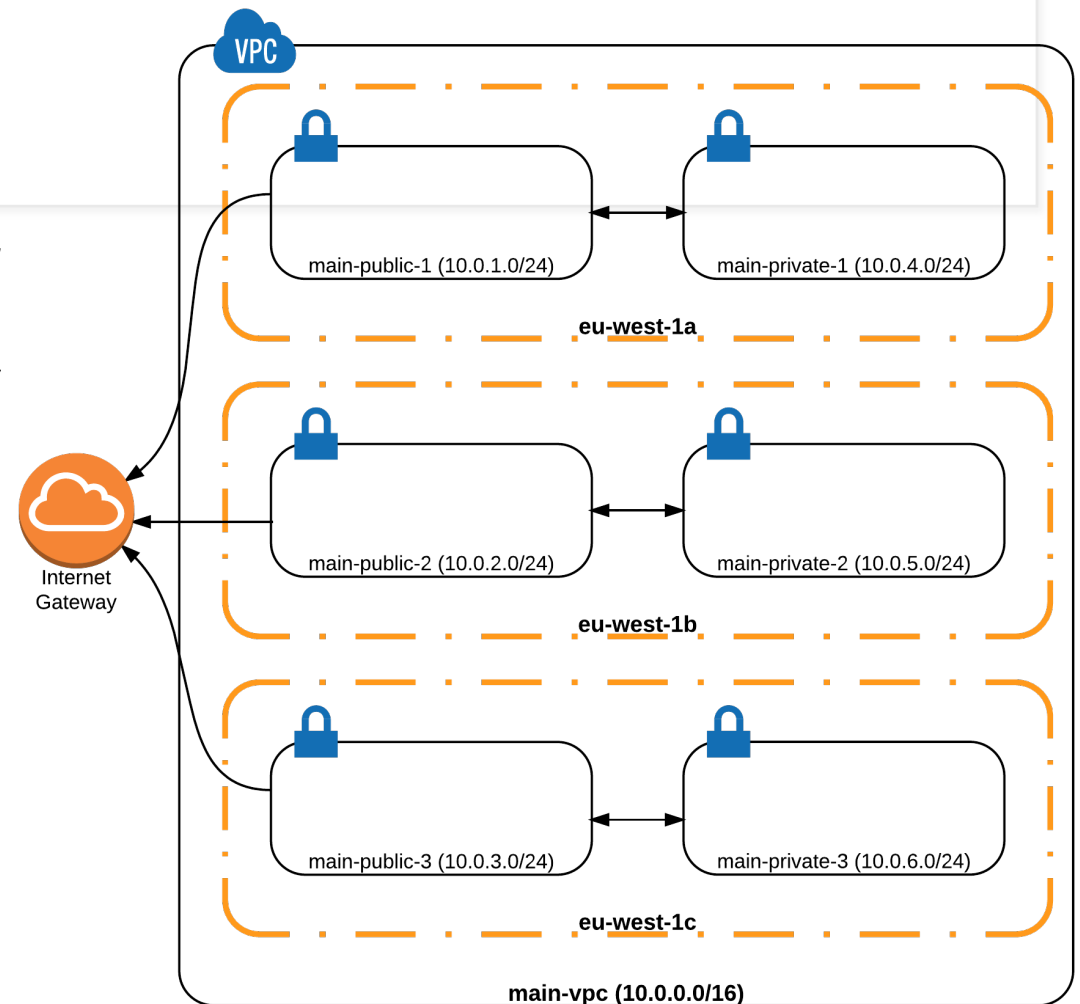
- Sur Amazon AWS, vous avez un VPC (Virtual Private Network) par défaut créé pour vous par AWS pour lancer des instances dedans
- Jusqu'à présent, nous utilisons ce VPC par défaut
- VPC isole les instances au niveau du réseau
  - C'est comme votre propre réseau privé dans le cloud
- La meilleure pratique consiste à toujours lancer vos instances dans un VPC
  - le VPC par défaut
  - ou un que vous créez vous-même (géré par terraform)

# Création de VPC

- Il y a aussi EC2-Classic, qui est essentiellement un grand réseau où tous les clients AWS pourraient lancer leurs instances dedans
- Pour les configurations petites à moyennes, **un VPC** (par région) conviendra pour vos besoins
- Une instance lancée dans un VPC ne peut jamais communiquer avec une instance dans un autre VPC en utilisant leurs **adresses IP privées**
  - Ils pouvaient encore communiquer, mais en utilisant leur IP publique (pas conseillé)
  - Vous pouvez également lier 2 VPC, appelé **peering**

# Création de VPC

- Sur Amazon AWS, vous commencez par créer votre propre Réseau Privé Virtuel pour déployer vos instances (serveurs) / bases de données
- Ce VPC utilise l'espace d'adressage 10.0.0.0/16, ce qui vous permet d'utiliser les adresses IP qui commencent par « 10.0. », comme ceci : 10.0.x.x.
- Ce VPC couvre la région eu-west-1, qui est une région Amazon AWS en Irlande.



# Private Subnets

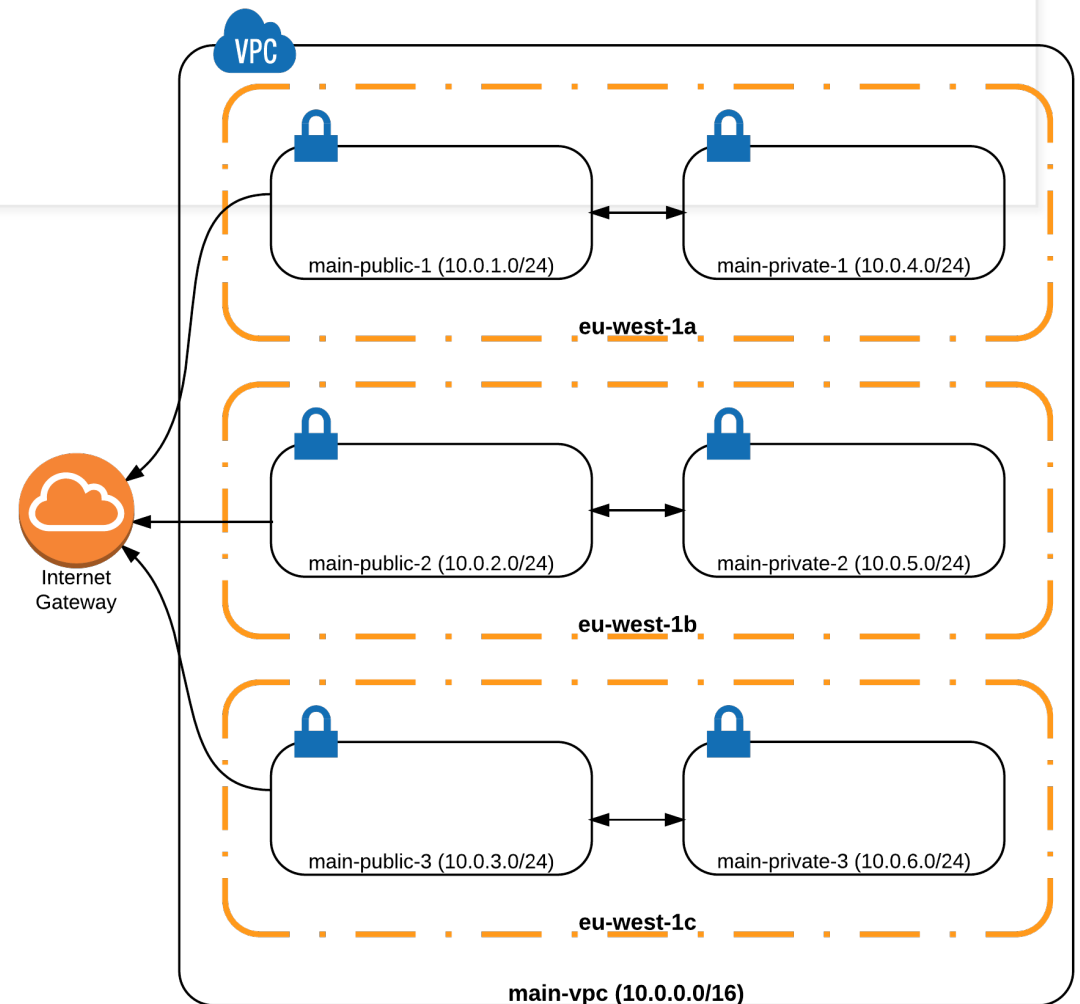
| Plage          | de          | à               |
|----------------|-------------|-----------------|
| 10.0.0.0/8     | 10.0.0.0    | 10.255.255.255  |
| 172.16.0.0/12  | 172.16.0.0  | 172.31.255.255  |
| 192.168.0.0/16 | 192.168.0.0 | 192.168.255.255 |

# Subnet Masks

| Range       | Network mask    | Total addresses | Description                                   | Exemples                                                  |
|-------------|-----------------|-----------------|-----------------------------------------------|-----------------------------------------------------------|
| 10.0.0.0/8  | 255.0.0.0       | 16,777,214      | Full 10.x.x.x range                           | <b>10.0.0.1</b> ,<br><b>10.100.200.20</b>                 |
| 10.0.0.0/16 | 255.255.0.0     | 65,536          | What we used for our VPC                      | <b>10.0.5.1</b> , <b>10.0.20.2</b> ,<br><b>10.0.100.3</b> |
| 10.1.0.0/16 | 255.255.0.0     | 65,536          | What we can use for another VPC               | <b>10.1.5.1</b> , <b>10.1.20.2</b> ,<br><b>10.1.100.3</b> |
| 10.0.0.0/24 | 255.255.255.0   | 256             | All addresses within from 10.0.0.0-10.0.0.255 | <b>10.0.0.1</b> , <b>10.0.0.2</b> ,<br><b>10.0.0.3</b>    |
| 10.0.1.0/24 | 255.255.255.0   | 256             | All addresses within from 10.0.1.0-10.0.1.255 | <b>10.0.1.1</b> , <b>10.0.1.2</b> ,<br><b>10.0.1.3</b>    |
| 10.0.0.5/32 | 255.255.255.255 | 1               | Single host                                   | <b>10.0.0.5</b>                                           |

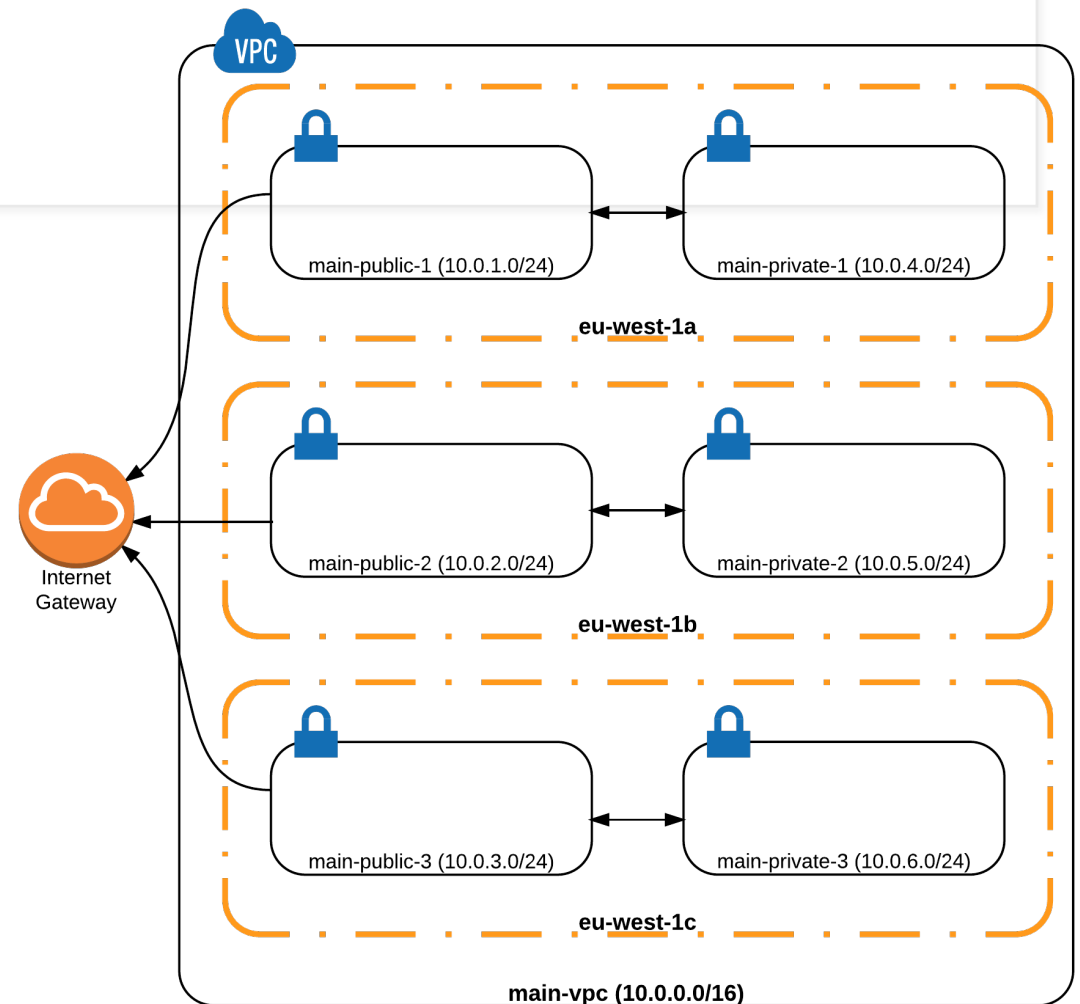
# Création de VPC

- Chaque **availability zone** (Amazon appelle un Datacenter une zone de disponibilité) a son propre sous-réseau public et privé
- Les instances démarrées dans le sous-réseau **mainpublic-3** auront l'adresse IP **10.0.3.x** et seront lancées dans la zone de disponibilité **eu-west-1c**
- Une instance lancée dans **mainprivate-1** aura une adresse IP **10.0.4.x** et résidera dans la zone de disponibilité (AZ) **eu-west-1a** d'Amazon.



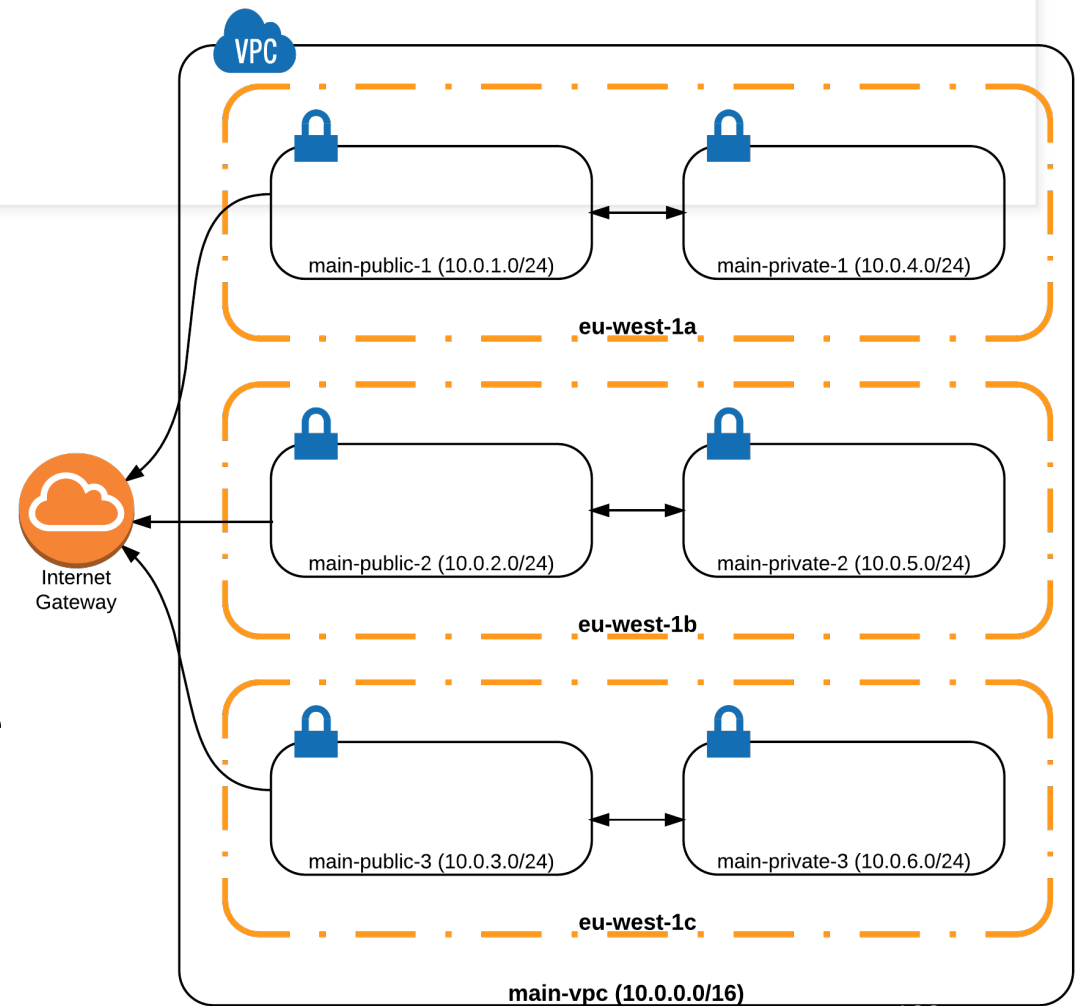
# Création de VPC

- Tous les sous-réseaux publics sont connectés à **une passerelle Internet**. Celles-ci les instances auront également une adresse IP publique, leur permettant d'être accessible depuis internet
- Les instances lancées dans les sous-réseaux privés n'obtiennent pas d'adresse IP publique, elles ne seront donc pas accessibles depuis Internet.
- Les instances de main-public peuvent atteindre des instances de main-private, car elles se trouvent toutes dans le même VPC. Cela bien sûr si vous configurez le pare-feu pour autoriser le trafic de l'un à l'autre.



# Création de VPC

- En règle générale, vous utilisez les sous-réseaux publics pour les services/applications accessibles sur Internet
- Les bases de données, les services de mise en cache et les backends vont tous dans les sous-réseaux privés
- Si vous utilisez un équilibreur de charge (LB), vous placerez généralement le LB dans les sous-réseaux publics et les instances servant une application dans les sous-réseaux privés.





# Atelier

- Lab
- AWS VPC